

GPUMODE Lecture Study Notes: Lecture 96: TLX

Core Problem the Lecture Addresses

This lecture presents **TLX**, also described as a **Triton low-level language extension**. The central problem is that ordinary Triton gives programmers a very productive tile-level programming model, but many of the highest-performance kernels on modern NVIDIA GPUs require explicit control over scheduling, pipelining, warp specialization, shared memory, distributed shared memory, and persistent execution.

The lecture's key claim is:

Triton is excellent when the compiler can infer the right low-level schedule from a tile-level program. TLX extends Triton by letting the user explicitly control tile scheduling and memory movement while still avoiding raw thread-level CUDA programming.

The problem is not that Triton cannot express computation. It can express tensor and tile computation very well. The problem is that on modern GPUs such as Hopper, Blackwell, and future architectures, peak performance depends on decisions such as:

- which warp group performs memory movement;
- which warp group performs matrix multiply-accumulate operations;
- which warp group performs epilogue work;
- how many pipeline stages are active;
- how shared memory buffers are allocated and reused;
- whether multiple CTAs cooperate on one logical tile;
- whether a kernel is persistent or non-persistent;
- whether work is statically assigned or dynamically stolen using cluster launch control;

- whether global memory is moved directly into shared memory using asynchronous copies or TMA.

In ordinary Triton, many of these choices are implicit compiler decisions. This works well for many simple kernels, especially simple matrix multiplication kernels, but becomes brittle for complex kernels such as FlashAttention, layer normalization, grouped GEMM, and fused production kernels. TLX therefore attempts to occupy a middle layer:

Do not expose raw CUDA threads and lane layouts to the user, but do expose enough tile-level control to let the user build high-performance schedules.

Required Background

Triton’s Existing Tile-Centric Model

Triton’s fundamental abstraction is that the programmer writes a program over **tiles** rather than individual CUDA threads. A tile may represent a vector, matrix block, or tensor slice. The compiler then lowers the tile-level operations into GPU execution involving threads, warps, registers, shared memory, vectorized memory accesses, and hardware instructions.

A simplified Triton-style mental model is:

The user writes what a tile computes. The compiler decides how individual threads inside the tile perform the work.

This abstraction hides:

- per-thread ownership of elements;
- lane-level indexing;
- shared memory swizzling details;
- vectorization details;
- memory coalescing mechanics;
- many synchronization decisions.

This is powerful because it makes kernels easier to write. However, as kernels become more schedule-sensitive, hiding all scheduling details becomes a limitation.

GPU Execution Model

A CUDA-style GPU program is organized into a hierarchy:

- **Grid**: collection of thread blocks.

- **CTA or thread block:** a group of threads scheduled together on one streaming multiprocessor.
- **Warp:** usually 32 threads executing in SIMD-style lockstep.
- **Warp group:** a collection of warps used by modern tensor-core programming models.
- **SM:** streaming multiprocessor containing warp schedulers, registers, shared memory, tensor cores, CUDA cores, and load/store units.

A common indexing model is:

$$\text{global_index} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

Triton and TLX move the user away from this explicit thread formula. Instead of asking which thread owns which scalar, TLX asks which tile, CTA, or warp group owns which logical phase of the pipeline.

Memory Hierarchy

The relevant memory hierarchy is:

- **Registers:** fastest storage, private to each thread.
- **Shared memory:** programmer-controlled memory local to an SM.
- **Distributed shared memory:** shared memory visible across CTAs in a cluster.
- **L1 cache:** close cache, often coupled with shared memory resources.
- **L2 cache:** chip-wide cache shared across SMs.
- **Global memory:** high-capacity HBM, high bandwidth but high latency.
- **TMEM:** tensor memory used by Blackwell-style tensor core workflows.

A useful ordering is:

register latency < shared memory latency < L1/L2 latency < global memory latency.

The lecture repeatedly uses this memory hierarchy to explain why TLX exposes shared memory control: when a value cannot remain in registers for the whole kernel, spilling to shared memory can be far better than spilling to global memory.

Arithmetic Intensity and the Roofline Model

A kernel's arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs executed}}{\text{bytes moved from memory}}.$$

A simple roofline bound is:

$$P \leq \min(P_{\max}, B_{\max} \cdot I),$$

where:

- P is achieved performance in FLOPs per second;
- $P_{\max}$ is peak compute throughput;
- $B_{\max}$ is peak memory bandwidth;
- I is arithmetic intensity.

A kernel is **memory-bound** when:

$$B_{\max} \cdot I < P_{\max}.$$

A kernel is **compute-bound** when:

$$B_{\max} \cdot I \geq P_{\max}.$$

TLX optimizations often increase effective arithmetic intensity by:

- increasing tile size;
- reusing operands from shared memory;
- avoiding redundant global loads;
- using persistent kernels to reduce launch and prologue overhead;
- increasing tensor core utilization.

Main Concepts

What TLX Adds to Triton

Triton asks the user to describe:

what each tile computes.

TLX asks the user to additionally describe:

how tiles are scheduled, synchronized, buffered, and pipelined.

The lecture emphasizes that TLX is not a replacement for Triton. It is an extension that keeps the tile-centric model while exposing schedule-level control.

TLX exposes control over:

- **pipelining**: multiple stages of memory movement and computation;
- **warp specialization**: producer, consumer, and epilogue warp groups;
- **shared memory management**: explicit local buffers;
- **distributed shared memory**: cross-CTA communication inside a cluster;
- **dynamic launch control**: hardware-assisted work stealing on Blackwell;
- **two-CTA tensor core mode**: cooperation between paired CTAs;
- **persistent scheduling**: one CTA or CTA group handles multiple output tiles.

However, TLX still avoids exposing:

- individual CUDA thread layout;
- lane-level ownership;
- exact shared memory offsets;
- raw swizzling details;
- manual per-thread synchronization.

The Tile-Level Mental Model

A helpful TLX mental model is:

Each tile behaves like a coarse logical thread. The user coordinates tiles and tile buffers, while the compiler maps the tile operations to actual GPU threads and memory layouts.

This differs from CUDA, where the programmer often writes:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

In TLX, the programmer instead thinks in terms of objects such as:

- a tile of A ;
- a tile of B ;

- an accumulator tile;
- a shared memory buffer for an input tile;
- a barrier protecting one pipeline stage;
- a CTA cluster handling a logical row or output tile.

Why Modern GPUs Pressure Triton’s Original Model

The lecture argues that modern GPU performance is not only about computation. It is about **who does the computation, when they do it, and how data moves through the pipeline.**

For example, a GEMM kernel may involve:

- TMA loading A and B tiles from global memory;
- waiting for those tiles to arrive in shared memory;
- issuing tensor core MMA operations;
- accumulating into TMEM;
- moving TMEM data into registers or shared memory;
- applying epilogue operations;
- storing the final output to global memory.

If these are executed serially, the timeline is:

$$\text{load}_0 \rightarrow \text{MMA}_0 \rightarrow \text{load}_1 \rightarrow \text{MMA}_1 \rightarrow \text{epilogue} \rightarrow \text{store}.$$

The desired timeline is closer to:

$$\begin{array}{lll} \text{load}_{k+1} & \text{overlaps with} & \text{MMA}_k, \\ \text{epilogue}_t & \text{overlaps with} & \text{MMA}_{t+1}, \\ \text{prologue}_{t+1} & \text{overlaps with} & \text{epilogue}_t. \end{array}$$

This scheduling control is difficult to infer perfectly from a purely high-level Triton program.

Hardware-Level Explanation

Warp Specialization

Warp specialization assigns different warp groups to different roles. A typical tensor-core pipeline has:

- **producer warp group:** loads A and B tiles into shared memory;

- **consumer or MMA warp group:** performs tensor core matrix multiplication;
- **epilogue warp group:** reads accumulated results, applies post-processing, and stores output.

A conceptual schedule is:

Time	Producer	MMA Consumer	Epilogue
0	load tile 0	idle	idle
1	load tile 1	MMA tile 0	idle
2	load tile 2	MMA tile 1	idle
3	load tile 3	MMA tile 2	epilogue tile 0

The goal is to keep hardware engines busy:

$$\text{TMA utilization} \approx 1, \quad \text{tensor core utilization} \approx 1.$$

Barriers as Tile-Level Synchronization

TLX uses barriers to coordinate tile-level phases. A producer fills a buffer and signals that it is full. A consumer waits for the buffer, performs computation, then signals that the buffer is empty.

A double-direction barrier protocol has two logical states:

$$\text{empty} \rightarrow \text{full} \rightarrow \text{empty}.$$

Producer protocol:

wait until buffer is empty,
load data into buffer,
signal buffer is full.

Consumer protocol:

wait until buffer is full,
consume data from buffer,
signal buffer is empty.

This prevents race conditions such as:

producer overwrites buffer s while consumer still reads buffer s .

Shared Memory and Register Pressure

Layer normalization is used as a case study because it stresses register capacity. If a row has N elements and each element occupies b bytes, then storing the full row requires:

$$N \cdot b$$

bytes. For $N = 65536$ and $b = 4$, this is:

$$65536 \cdot 4 = 262144 \text{ bytes.}$$

This exceeds the convenient register capacity of a single CTA and can also be large relative to per-CTA shared memory. Therefore, a naive implementation reloads the row several times.

If an input row x is used for mean, variance, and final normalization, a naive implementation may load:

$$3N \cdot b$$

bytes from global memory or cache. If TLX stores x in shared memory once and reloads from shared memory, global traffic can be closer to:

$$N \cdot b$$

for the input row, plus loads of weights and biases.

Distributed Shared Memory

Distributed shared memory allows CTAs inside a cluster to communicate through shared memory. Suppose a reduction over a row is split across C CTAs. CTA c computes a partial sum:

$$s_c = \sum_{j \in \mathcal{J}_c} x_j,$$

where $\mathcal{J}_c$ is the subset of columns owned by CTA c . The final sum is:

$$s = \sum_{c=0}^{C-1} s_c.$$

A TLX-style cross-CTA reduction has three phases:

1. each CTA computes its local partial reduction;
2. each CTA stores its partial result into the remote slots of the other CTAs;
3. each CTA locally reduces the C slots to obtain the same final reduction value.

The resulting shared memory layout per CTA can be viewed as:

$$\text{slots} = [s_0, s_1, \dots, s_{C-1}].$$

After remote stores complete, every CTA has all partial values and can compute:

$$s = \sum_{c=0}^{C-1} \text{slots}[c].$$

Asynchronous Global-to-Shared Copies

The lecture distinguishes several memory movement operations:

- **local load:** shared memory to registers;
- **local store:** registers to shared memory;
- **async load:** global memory directly to shared memory;
- **TMA load:** tensor memory accelerator load, usually stricter but more efficient for aligned tensor tiles.

A synchronous path may look like:

$$\text{global memory} \rightarrow \text{registers} \rightarrow \text{shared memory}.$$

An asynchronous direct path is:

$$\text{global memory} \rightarrow \text{shared memory}.$$

This reduces intermediate register pressure and allows other work to proceed while memory movement is in flight.

Mathematical Reasoning

Layer Normalization

For an input row $x \in \mathbb{R}^N$, layer normalization computes:

$$\mu = \frac{1}{N} \sum_{j=0}^{N-1} x_j.$$

The variance is:

$$\sigma^2 = \frac{1}{N} \sum_{j=0}^{N-1} (x_j - \mu)^2.$$

The inverse standard deviation is:

$$r = \frac{1}{\sqrt{\sigma^2 + \epsilon}},$$

where ϵ is a small numerical stability constant.

The normalized output with weight γ_j and bias β_j is:

$$y_j = (x_j - \mu)r\gamma_j + \beta_j.$$

A direct implementation touches x at least three times:

pass 1: compute μ ,
pass 2: compute σ^2 ,
pass 3: compute y_j .

If each pass loads x from global memory, the input traffic is:

$$T_{\text{global}} = 3Nb.$$

If x is loaded once into shared memory and then reused, global input traffic becomes:

$$T_{\text{global}} = Nb,$$

while shared memory traffic increases. This is usually beneficial because shared memory latency is much lower than global memory latency.

Reduction Split Across CTAs

If C CTAs cooperate on a row, each CTA handles approximately:

$$\frac{N}{C}$$

elements. The local partial mean sum is:

$$s_c = \sum_{j=cN/C}^{(c+1)N/C-1} x_j.$$

The global mean is:

$$\mu = \frac{1}{N} \sum_{c=0}^{C-1} s_c.$$

Similarly, after μ is known, each CTA computes a partial variance sum:

$$v_c = \sum_{j \in \mathcal{J}_c} (x_j - \mu)^2,$$

and the final variance is:

$$\sigma^2 = \frac{1}{N} \sum_{c=0}^{C-1} v_c.$$

GEMM

For matrix multiplication:

$$C = AB,$$

with:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

each element is:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

A tiled GEMM partitions the computation into blocks:

$$C_{I,J} = \sum_p A_{I,p} B_{p,J},$$

where I indexes a block of rows, J indexes a block of columns, and p indexes a block of the reduction dimension.

The arithmetic work for a tile of shape $B_M \times B_N$ over reduction block B_K is approximately:

$$2B_M B_N B_K$$

FLOPs, counting one multiply and one add as two floating-point operations.

The data movement per K -tile is approximately:

$$b(B_M B_K + B_K B_N),$$

where b is bytes per operand element. The tile arithmetic intensity is therefore:

$$I_{\text{tile}} = \frac{2B_M B_N B_K}{b(B_M B_K + B_K B_N)} = \frac{2B_M B_N}{b(B_M + B_N)}.$$

Increasing B_M and B_N improves reuse and increases arithmetic intensity, but also increases shared memory, register, and accumulator pressure.

Pipeline Depth

If a pipeline has S stages, then a buffer index can be computed as:

$$s(k) = k \bmod S.$$

At iteration k , the producer fills buffer $s(k)$. The consumer must ensure that buffer $s(k)$ has been released from the previous use before overwriting it.

The rough steady-state latency hiding condition is:

$$S \cdot T_{\text{compute}} \gtrsim T_{\text{load}},$$

where:

- S is the number of pipeline stages;
- T_{compute} is the compute time per tile;
- T_{load} is the memory load latency or effective load service time.

If S is too small, memory latency is visible. If S is too large, shared memory usage becomes too high and occupancy may fall.

Persistent Kernel Benefit

A non-persistent kernel assigns one CTA to one output tile. The timeline has repeated prologue and epilogue gaps:

prologue $\rightarrow$ steady compute $\rightarrow$ epilogue $\rightarrow$ new CTA prologue.

A persistent kernel lets the same CTA process multiple tiles:

prologue₀ $\rightarrow$ steady tile₀ $\rightarrow$ steady tile₁ $\rightarrow \dots \rightarrow$ epilogue _{$T-1$} .

If one CTA processes T tiles, the amortized overhead per tile becomes:

$$\frac{T_{\text{prologue}} + T_{\text{epilogue}}}{T}.$$

As T increases, this overhead becomes small.

Code Walkthrough

Representative LayerNorm in Ordinary Triton Style

The lecture describes a Triton-style layer normalization that loops over the row multiple times because a large row cannot fit entirely in registers.

Representative pseudocode, not exact lecture source.

```
for row in rows:
    # Pass 1: compute mean.
    sum_x = 0.0
    for block in blocks_along_N:
        x = load(X[row, block])
        sum_x += reduce_sum(x)
    mean = sum_x / N

    # Pass 2: compute variance.
    sum_var = 0.0
    for block in blocks_along_N:
        x = load(X[row, block])
        diff = x - mean
        sum_var += reduce_sum(diff * diff)
    var = sum_var / N
```

```

inv_std = rsqrt(var + eps)

# Pass 3: normalize.
for block in blocks_along_N:
    x = load(X[row, block])
    gamma = load(W[block])
    beta = load(B[block])
    y = (x - mean) * inv_std * gamma + beta
    store(Y[row, block], y)

```

The important hardware behavior is:

- X is loaded repeatedly.
- Cache may help, but the kernel still executes the load instructions.
- If N is large, keeping all of x in registers is infeasible.
- Register pressure can cause spills.
- Ordinary Triton does not give the user direct shared memory allocation control for this pattern.

TLX-Style LayerNorm with Shared Memory Reuse

A TLX version can explicitly buffer x in shared memory.

Representative TLX-like pseudocode.

```

x_smem = alloc_shared(shape=(BLOCK_M, BLOCK_N_PER_CTA),
                       dtype=float32)

```

Load input once into shared memory.

```

async_load_global_to_shared(
    src=X[row_offsets, col_offsets],
    dst=x_smem
)
commit_async_loads()
wait_async_loads()

```

Mean reduction.

```

x = local_load(x_smem)
partial_sum = reduce_sum(x)
mean = cluster_reduce_sum(partial_sum) / N

```

Variance reduction.

```

x = local_load(x_smem)
diff = x - mean
partial_var = reduce_sum(diff * diff)

```

```
var = cluster_reduce_sum(partial_var) / N
inv_std = rsqrt(var + eps)
```

```
# Normalization.
x = local_load(x_smem)
gamma = load(W[col_offsets])
beta = load(B[col_offsets])
y = (x - mean) * inv_std * gamma + beta
store(Y[row_offsets, col_offsets], y)
```

The major difference is that x has a short register lifetime each time it is loaded from shared memory. Instead of keeping x live across mean, variance, and normalization, the program reloads it cheaply from shared memory.

The effective register lifetimes are:

```
 $x_{\text{mean}}$  : live only during mean pass,
 $x_{\text{var}}$  : live only during variance pass,
 $x_{\text{norm}}$  : live only during normalization pass.
```

This reduces register pressure.

Distributed Shared Memory Reduction

The lecture describes a cross-CTA reduction where each CTA writes its partial result into the remote shared memory slots of other CTAs.

```
# Representative TLX-like pseudocode.
```

```
num_ctas = CLUSTER_SIZE
```

```
partial_slots = alloc_shared(shape=(num_ctas, BLOCK_M),
                             dtype=float32)
```

```
barrier = alloc_barrier()
```

```
# Each CTA computes its local partial reduction.
```

```
x_local = local_load_or_global_load(...)
partial = reduce_sum(x_local)
```

```
# Store local partial into own slot.
```

```
local_store(partial_slots[cta_id], partial)
```

```
# Expect remote writes from all other CTAs.
```

```
barrier.expect_bytes((num_ctas - 1) * BLOCK_M * sizeof(float32))
```

```
# Send this CTA's partial result to the slots of peer CTAs.
```

```

for peer in range(num_ctas):
    if peer != cta_id:
        async_remote_store(
            dst=remote(partial_slots[cta_id], peer),
            src=partial,
            barrier=remote(barrier, peer)
        )

# Wait until all remote writes into this CTA's slots are complete.
barrier.wait()

# Now every CTA has all partials locally.
all_partials = local_load(partial_slots)
final = reduce_sum(all_partials)

```

The user thinks in terms of partial tiles and slots. The compiler decides which actual threads perform the stores and how the tile-level data maps into registers and shared memory.

Representative GEMM Without Pipelining

A baseline tile-level GEMM is conceptually:

```

# Representative non-pipelined GEMM pseudocode.

acc = zeros((BLOCK_M, BLOCK_N), dtype=float32)

for k0 in range(0, K, BLOCK_K):
    a = tma_load(A[m_offsets, k0 : k0 + BLOCK_K])
    b = tma_load(B[k0 : k0 + BLOCK_K, n_offsets])

    wait_for_tma(a)
    wait_for_tma(b)

    acc += mma(a, b)

c = epilogue(acc)
store(C[m_offsets, n_offsets], c)

```

This is inefficient because the timeline is serial:

TMA load → wait → MMA → next TMA load.

The tensor cores may be idle during loads, and the TMA engine may be idle during MMA.

GEMM with Warp Specialization and Pipeline Buffers

A TLX-style pipelined GEMM allocates multiple buffers and uses barriers between producer and consumer warp groups.

Representative TLX-like GEMM pseudocode.

```
STAGES = 3

a_smem = alloc_shared(shape=(STAGES, BLOCK_M, BLOCK_K),
                       dtype=bf16)
b_smem = alloc_shared(shape=(STAGES, BLOCK_K, BLOCK_N),
                       dtype=bf16)
acc_tmem = alloc_tmem(shape=(BLOCK_M, BLOCK_N),
                      dtype=float32)

bar_full = alloc_barrier(shape=(STAGES,))
bar_empty = alloc_barrier(shape=(STAGES,))
bar_acc_done = alloc_barrier(shape=(1,))

if warp_group_role == "producer":
    for k_iter in range(num_k_tiles):
        s = k_iter % STAGES

        bar_empty[s].wait()

        tma_load(
            dst=a_smem[s],
            src=A_tile(k_iter),
            barrier=bar_full[s]
        )
        tma_load(
            dst=b_smem[s],
            src=B_tile(k_iter),
            barrier=bar_full[s]
        )

elif warp_group_role == "mma":
    acc = acc_tmem

    for k_iter in range(num_k_tiles):
        s = k_iter % STAGES

        bar_full[s].wait()

        acc = mma(acc, a_smem[s], b_smem[s])
```

```

        bar_empty[s].arrive()

    bar_acc_done.arrive()

elif warp_group_role == "epilogue":
    bar_acc_done.wait()

    c = load_accumulator(acc_tmem)
    c = apply_epilogue(c)
    store(C_tile, c)

```

The key hardware effects are:

- producer warp group issues TMA loads;
- MMA warp group issues tensor core operations;
- epilogue warp group handles output processing;
- barriers prevent reading unfilled buffers or overwriting live buffers;
- multiple stages allow memory movement to overlap with tensor core work.

Persistent GEMM with Cluster Launch Control

The lecture discusses **cluster launch control**, abbreviated as CLC or CRC in the transcript. The relevant idea is that Blackwell provides hardware support for dynamically fetching CTA IDs corresponding to work that has not started yet.

A representative persistent loop is:

```

# Representative persistent GEMM pseudocode.

ctx = clc_create_context(num_consumers=3)

while True:
    work = clc_fetch_next(ctx)

    if work.is_empty():
        break

    tile_m, tile_n = decode_work(work)

    run_pipelined_gemm_tile(tile_m, tile_n)

```

The difference from static persistent scheduling is:

- static persistent kernels predefine which CTA handles which tiles;
- CLC dynamically fetches the next available tile at runtime;

- this adapts to real-time SM availability;
- if some SMs are occupied by other work, remaining SMs can still steal work.

This improves load balancing.

Two-CTA GEMM Mode

The lecture explains a two-CTA mode where two CTAs cooperate on a tensor core computation. The main benefit is that operand B can be split across the two CTAs rather than duplicated.

In a conventional two-CTA non-paired setup:

$$\begin{aligned}\text{CTA}_0 &: A_0, B, \\ \text{CTA}_1 &: A_1, B.\end{aligned}$$

Each CTA has its own copy of B in shared memory. In paired two-CTA mode:

$$\begin{aligned}\text{CTA}_0 &: A_0, B_0, \\ \text{CTA}_1 &: A_1, B_1,\end{aligned}$$

where:

$$B = [B_0, B_1].$$

The hardware tensor core mechanism can access distributed shared memory across the paired CTAs. This reduces shared memory usage for B by approximately:

$$\frac{1}{2}.$$

Representative logic:

Representative two-CTA synchronization logic.

```
if paired_cta_mode:
    if cta_id == 1:
        # Signal CTA 0 that CTA 1's B half is ready.
        remote_arrive(barrier_on_cta0)

    if cta_id == 0:
        # CTA 0 waits for CTA 1's B half.
        local_wait(barrier_on_cta0)
```

```

# CTA 0 issues the paired MMA instruction.
# Hardware uses tensor core resources from both CTAs.
paired_mma(acc, a_smem, b_distributed_smem)

# MMA completion can signal barriers for both CTAs.

```

The user must express the synchronization relationship, but not the internal thread-level tensor core layout.

Data Partitioning

When tile sizes become large, accumulator storage or TMEM capacity can become limiting. Data partitioning splits a large output tile into subtiles.

For example, a 256×256 output tile can be split into two 128×256 subtiles:

$$C = \begin{bmatrix} C_0 \\ C_1 \end{bmatrix}.$$

The schedule can then overlap:

$$\text{epilogue}(C_0) \quad \text{with} \quad \text{MMA}(C_1).$$

Representative pseudocode:

```

# Representative data-partitioned GEMM pseudocode.

```

```

NUM_SUBTILES = 2

```

```

for k_iter in range(num_k_tiles):
    for subtile in static_range(NUM_SUBTILES):
        load_a_for_subtile(subtile, k_iter)
        load_b_for_tile(k_iter)

        for subtile in static_range(NUM_SUBTILES):
            wait_for_operands(subtile)
            mma_subtile(subtile)

for subtile in static_range(NUM_SUBTILES):
    wait_for_accumulator(subtile)
    epilogue_and_store(subtile)

```

The main point is not just smaller storage. The main point is that subtiles create more scheduling opportunities.

Epilogue Subtiling

The epilogue may need shared memory to move data from TMEM to global memory. If it uses the entire output tile at once, shared memory pressure can be high. Subtiling the epilogue processes only part of the output tile at a time.

For a tile $C \in \mathbb{R}^{256 \times 256}$, the epilogue may process:

$$256 \times 32$$

columns at a time, rather than:

$$256 \times 256.$$

This reduces epilogue shared memory use by:

$$\frac{256 \cdot 32}{256 \cdot 256} = \frac{1}{8}.$$

Representative pseudocode:

Representative epilogue subtiling pseudocode.

```
for n0 in range(0, BLOCK_N, EPILOGUE_TILE_N):
    c_part = tmem_load(
        acc_tmem[:, n0 : n0 + EPILOGUE_TILE_N]
    )

    c_part = apply_epilogue(c_part)

    local_store(epilogue_smem, c_part)

    store_global(
        C[:, n0 : n0 + EPILOGUE_TILE_N],
        epilogue_smem
    )
```

This frees shared memory for producer buffers and can allow larger GEMM tiles or deeper pipelines.

Performance Intuition

Why TLX Helps LayerNorm

LayerNorm is dominated by reductions and memory movement. It does not use tensor cores. Its bottlenecks include:

- repeated reads of x ;
- register pressure from keeping x live;
- reduction synchronization;
- limited per-CTA shared memory;
- memory latency.

TLX helps by enabling:

- explicit shared memory buffering of x ;
- async global-to-shared loads;
- distributed shared memory across CTAs;
- shorter register lifetimes;
- cross-CTA reductions without dropping to raw CUDA.

The performance improvement comes from trading expensive global memory reloads and register spills for cheaper shared memory reloads.

Why TLX Helps GEMM

GEMM is usually tensor-core dominated. Peak performance requires:

- high tensor core utilization;
- enough tile reuse;
- enough pipeline stages;
- efficient TMA usage;
- overlapping prologue, mainloop, and epilogue;
- avoiding shared memory exhaustion;
- avoiding TMEM bottlenecks;
- load balancing across SMs.

TLX helps GEMM because it lets the programmer explicitly compose:

- warp specialization;
- TMA producer pipelines;
- MMA consumer pipelines;
- epilogue overlap;
- persistent loops;
- CLC-based dynamic work stealing;

- two-CTA paired tensor core mode;
- data partitioning;
- epilogue subtiling.

Why Tile Size Matters

For GEMM, larger tiles increase operand reuse. The tile arithmetic intensity derived earlier is:

$$I_{\text{tile}} = \frac{2B_M B_N}{b(B_M + B_N)}.$$

If $B_M = B_N = B$, then:

$$I_{\text{tile}} = \frac{2B^2}{2bB} = \frac{B}{b}.$$

Thus, larger square tiles increase arithmetic intensity linearly in B . However, larger tiles also increase:

- shared memory use;
- accumulator use;
- TMEM use;
- register pressure;
- synchronization complexity.

TLX gives enough control to exploit large tiles while managing these costs.

Why Persistent Kernels Reduce Bubbles

In a non-persistent kernel, the first load and final store of each CTA may not overlap well with neighboring CTAs if occupancy is limited by shared memory. This produces tensor core bubbles.

If the mainloop time per tile is T_{main} , prologue time is T_{pro} , and epilogue time is T_{epi} , then non-persistent tile time is approximately:

$$T_{\text{nonpersistent}} = T_{\text{pro}} + T_{\text{main}} + T_{\text{epi}}.$$

With a persistent loop over Q tiles, the average time is closer to:

$$T_{\text{persistent}} \approx T_{\text{main}} + \frac{T_{\text{pro}} + T_{\text{epi}}}{Q}.$$

The larger Q becomes, the less visible the prologue and epilogue become.

Why Dynamic Work Stealing Matters

Static persistent scheduling can become imbalanced if:

- some tiles take longer than others;
- some SMs are occupied by other workloads;
- the grid shape does not divide evenly across SMs;
- runtime conditions differ from compile-time assumptions.

Cluster launch control lets active SMs dynamically fetch work. This approximates:

$$\text{next tile} = \arg \min_{\text{unfinished tiles}} \text{not yet started.}$$

The benefit is better utilization:

$$\text{SM utilization} = \frac{\text{active SM cycles}}{\text{total available SM cycles}}.$$

CLC improves the numerator by reducing idle SM cycles.

Common Misconceptions

Misconception: TLX Replaces Triton

TLX does not replace Triton. It extends Triton with lower-level scheduling and memory-control features. The tile abstraction remains central.

Misconception: TLX Exposes Raw CUDA Threads

TLX does not require the user to write explicit per-thread logic. The user still programs with tiles. Thread layout, lane ownership, and most low-level mapping details remain compiler-managed.

Misconception: Shared Memory Is Always Better

Shared memory is faster than global memory, but it is limited. Excessive shared memory use can reduce occupancy:

$$\text{resident CTAs per SM} \leq \left\lfloor \frac{S_{\text{SM}}}{S_{\text{CTA}}} \right\rfloor,$$

where S_{SM} is shared memory capacity per SM and S_{CTA} is shared memory used per CTA. If S_{CTA} is too large, occupancy may fall to one CTA per SM.

Misconception: More Pipeline Stages Always Improve Performance

More stages hide more latency but consume more shared memory:

$$S_{\text{total}} = S_{\text{stages}} \cdot S_{\text{per stage}}.$$

Too many stages can reduce occupancy or prevent larger tiles. Pipeline depth must be tuned.

Misconception: Larger Tiles Are Always Better

Larger tiles increase arithmetic intensity but also increase resource usage. The best tile depends on:

- tensor core shape;
- shared memory capacity;
- TMEM capacity;
- register pressure;
- number of pipeline stages;
- epilogue cost;
- problem shape.

Misconception: Compiler Heuristics Are Enough for All Kernels

Compiler heuristics work very well for many kernels. The motivation for TLX is that complex kernels often require user-visible scheduling decisions, especially when combining memory movement, tensor cores, reductions, fusion, and irregular workloads.

Practical Takeaways

1. Triton is excellent for tile-level computation, but modern GPU peak performance often requires explicit tile-level scheduling.
2. TLX exposes scheduling control without forcing the user into raw CUDA thread programming.
3. Shared memory management is critical for reducing register pressure in reduction-heavy kernels such as layer normalization.
4. Distributed shared memory enables CTA clusters to cooperate on reductions and data movement.

5. Asynchronous global-to-shared copies reduce register pressure and improve overlap.
6. Warp specialization is essential for high-performance tensor-core kernels on Blackwell-style architectures.
7. Producer-consumer barriers are the core abstraction for safe pipelining.
8. Persistent kernels reduce prologue and epilogue bubbles by letting one CTA process multiple tiles.
9. Cluster launch control improves load balancing through hardware-assisted dynamic work stealing.
10. Two-CTA paired mode can reduce shared memory usage for shared operands such as B in GEMM.
11. Data partitioning and epilogue subtiling are practical techniques for managing TMEM and shared memory pressure.
12. The best TLX kernel is still tunable: tile sizes, pipeline stages, CTA grouping, and epilogue granularity all matter.

Project or Experiment Ideas

Experiment 1: LayerNorm Reload Cost

Implement three versions of layer normalization:

1. naive PyTorch;
2. Triton-style multiple-pass row reduction;
3. CUDA or Triton-like version using explicit shared memory buffering.

Measure effective bandwidth:

$$B_{\text{eff}} = \frac{\text{bytes read} + \text{bytes written}}{\text{runtime}}.$$

Vary N and observe when register pressure and repeated loads become visible.

Experiment 2: Shared Memory Versus Register Pressure

Write a CUDA layer norm where the row is split into blocks. Compare:

- keeping partial data in registers;
- spilling through global memory;
- explicitly storing reusable data in shared memory.

Use Nsight Compute to inspect:

- register count;
- local memory traffic;
- shared memory throughput;
- occupancy;
- achieved memory bandwidth.

Experiment 3: Pipelined GEMM

Implement a simplified GEMM kernel with:

1. no software pipeline;
2. two-stage pipeline;
3. three-stage pipeline.

Measure:

$$\text{TFLOP/s} = \frac{2MNK}{t \cdot 10^{12}}.$$

Observe how pipeline stages improve performance until resource usage becomes limiting.

Experiment 4: Persistent Tile Scheduling

Create a kernel where each CTA processes multiple output tiles in a loop. Compare against one-CTA-per-tile scheduling.

Measure:

- runtime;
- SM occupancy;
- tail effects;
- tensor core utilization if using GEMM;
- load balancing across SMs.

Experiment 5: Epilogue Fusion

Start with a GEMM:

$$C = AB.$$

Then fuse an epilogue:

$$Y_{ij} = \text{gelu}(C_{ij} + b_j).$$

Compare:

- library GEMM plus separate epilogue kernel;
- fused custom kernel;
- Triton implementation;
- TLX-style conceptual design.

The key measurement is saved global memory traffic:

$$T_{\text{unfused}} \approx \text{write } C + \text{read } C + \text{write } Y,$$

while:

$$T_{\text{fused}} \approx \text{write } Y.$$

Final Mental Model

TLX should be understood as a schedule-aware extension of Triton. Triton asks the programmer to describe computation over tiles. TLX keeps that model but adds explicit control over how those tiles move through the GPU execution pipeline.

The final mental model is:

A high-performance modern GPU kernel is not just a formula. It is a carefully staged factory. Producer warp groups move data, consumer warp groups feed tensor cores, epilogue warp groups finish output, barriers protect buffers, shared memory controls reuse, persistent loops reduce bubbles, and dynamic launch control balances work across SMs. TLX gives the programmer access to this factory schedule while still hiding raw thread-level machinery.

Visual Concept

Draw a three-lane pipeline. The first lane is a producer warp group issuing TMA or async global-to-shared loads. The second lane is an MMA consumer warp group using tensor cores and TMEM. The third lane is an epilogue warp group storing results to global memory. Between lanes, draw shared memory buffers and full-empty barriers. Then draw multiple CTAs inside a cluster sharing distributed shared memory and dynamically fetching work from a global tile queue.